BCSE307L – COMPILER DESIGN PROJECT

PROJECT REVIEW 3

Final Demonstration, Evaluation and Project Completion Review

A24 – BURS-Based Retargetable Instruction Selection

Consolidated Final Report

Item	Details
Course Code / Title	Compiler Design – BCSE307L
Team Number	8
Project ID and Title	A24 – BURS-Based Retargetable Instruction Selection
Review Date	24/10/2026
Faculty / Evaluator	Dr. Shalini L
Team Member 1	Vineet Kumar Mishra – Reg. No. **INSERT**
Team Member 2	Mayank Verma – Reg. No. **INSERT**
Team Member 3	Utsav Keshrwani – Reg. No. 24BCE2434
Team Member 4	Ahmad Azim – Reg. No. 24BCT0119
Repository	https://github.com/ahmadazim542/burs-instruction-selector (public, branch `main`)
Maximum Marks	10

Declaration of Originality

We declare that this project report and the accompanying implementation are the work of Team 8, carried out for BCSE307L Compiler Design. All external material is acknowledged in Section 14: the BURG code-generator generator (third-party, used unmodified as a build tool), the published papers and textbook cited, and the AI coding assistant used during implementation and documentation. No part of this work has been submitted for any other course.

Vineet Kumar Mishra Mayank Verma Utsav Keshrwani

Ahmad Azim Date: _____

Contents

Section	Title	Section	Title
0	Review 2 Feedback Closure and Changes from Approved Plan	9	Testing Strategy and Test Cases
1	Abstract	10	Results and Evaluation
2	Introduction and Motivation	11	Error Handling, Limitations and Known Issues
3	Problem Statement, Objectives, Scope and Limitations	12	Member-Wise Contribution and Final Contribution Matrix
4	Background / Related Work	13	Conclusion and Future Work
5	Requirements and Technology Stack	14	References and Acknowledgements
6	System Design	A	User Manual and Installation Guide
7	Algorithms and Methodology	B	Sample Runs
8	Implementation Details	C	Grammar Specification (Target A and Target B)
		D	Review 3 Demonstration Plan, Checklist and Records

> **Reproducing every number in this report.** All results below were produced by the code in the repository on the team laptop (Apple M1, macOS 26.6, Apple clang 21) with `make clean && make && make test && make results`. The raw outputs are committed in `results/` and the expected test outputs in `tests/expected/`, so the evaluator can regenerate and compare them on any machine with a C compiler and `make`.

0. Review 2 Feedback Closure and Changes from Approved Plan

0.1 Closure of Review 2 action items

#	Review 2 observation / pending item	Action taken	Owner	Evidence	Status
1	Target-B rule table not yet defined	Defined a zero-address stack ISA (now 17 rules) with memory (`[var]`) and immediate (`#k`) operand forms	Mayank / Utsav	`src/targets/targetB/targetB.gr`; Section 6.4, App. C	Closed
2	Defect D3 – Target-B emitter not written	Implemented `reduceB()` and a Target-B simulator `simulateB()`	Ahmad	`targetB.gr`; tests in Section 9	Closed
3	Run all test expressions through Target B and record results	All six review expressions plus five extra (incl. division) run on both targets	Ahmad	Section 10.1; `results/sample_runs.txt`	Closed
4	Expand regression suite to both targets	31 stored regression cases + 11-expression suite + 7,140-tree exhaustive check	Ahmad	`tests/`; `results/test_report.txt`	Closed
5	Initialise team GitHub repository and push artefacts	Repository created with source, tests, results and all review documents	Ahmad	github.com/ahmadazim542/burs-instruction-selector	Closed
6	Folder structure to match the Review 1 proposed layout	`src/ir`, `src/burs`, `src/targets/targetA`, `src/targets/targetB`, `src/main`, `tests`, `docs`, `results`	Vineet	Section 8.1	Closed
7	Exact references (Review 1 placeholder)	Exact papers, editions and the BURG source cited	Mayank	Section 14	Closed
8	Register numbers on	Utsav and Ahmad inserted; Vineet	Vineet	Cover page	In

#	Review 2 observation / pending item	Action taken	Owner	Evidence	Status
	the cover page	and Mayank to be inserted			progress

0.2 Changes from the approved plan

Area	Approved plan (Review 1 / 2)	Final implementation	Justification
Language	Java / C++	C99 (single toolchain)	The BURG generator emits C; one language keeps IR, generated matchers and emitters in one build.
Code organisation	Single `burs.gr` holding IR, rules, emitter, tests	Separate IR, engine, per-target grammar and driver modules	Enforces the machine-independent / target-specific split that the project is about.
Target B	"Pending – Week 6"	17-rule stack machine with memory and immediate operands	Gives a genuinely different ISA (0-address vs 3-address), so retargeting is visible.
Verification	Expected outputs compared by hand	Built-in simulator per ISA + byte-exact regression + exhaustive check	Each generated program is executed and compared with the IR value automatically.
Evaluation	Not specified	Naive-selector baseline comparison and a scaling benchmark	Required by the Review 3 guidelines (comparison / performance evidence).
IR operators	CONST, VAR, ADD, SUB, MUL, LOAD	Same six plus DIV (with DIVI / DIVM operand forms on the targets)	Division is a natural extension requested during preparation; it exercises the retargeting claim: one IR change, one rule group per grammar, no engine change.

Scope is unchanged apart from one extension: the expression-tree IR now has CONST, VAR, ADD, SUB, MUL, DIV and LOAD; bottom-up matching with rule costs; two target ISAs; console harness.

1. Abstract

Instruction selection is the compiler back-end phase that maps machine-independent intermediate representation (IR) to the instructions of a particular processor. Hand-written selectors are hard to keep optimal and hard to move to a new processor. This project implements a compact, educational Bottom-Up Rewrite System (BURS) instruction selector that is retargetable by construction. An infix expression is parsed into an expression-tree IR with seven node types (CONST, VAR, ADD, SUB, MUL, DIV, LOAD). Each target machine is described by one tree grammar whose rules pair an IR pattern with a cost and an instruction template. The BURG code-generator generator compiles each grammar into a table-driven matcher; a common, machine-independent engine labels the tree bottom-up so that every node knows its cheapest rule for every nonterminal, backtracks the minimum-cost cover from the root, and drives the target's emitter in postorder. Two targets share the same IR and engine: Target A, a three-address load/store register machine (11 rules), and Target B, a zero-address stack machine with memory and immediate operand forms (17 rules). Every generated program is executed by a simulator for its ISA and compared with the IR's own evaluation. All 31 regression cases, an 11-expression review suite and an exhaustive check of 7,140 trees (13,692 target runs) pass with zero failures. Against a naive one-instruction-per-node selector under the same cost model, BURS lowers the average cover cost by 5.1% on Target A and 20.9% on Target B. Labeling runs in linear time, about 9–21 ns per node for trees of up to 240,000 nodes. The result is a reproducible demonstration that one matching engine can produce different, verified, cost-optimal code for two instruction sets by changing only the target description.

2. Introduction and Motivation

A compiler front end produces an IR that is independent of any processor. The back end must turn that IR into real instructions, and the choice is not unique: the expression `load(a) + 4` can be compiled on a register machine

as `LOAD`, `LOAD`, `LOADI`, `ADD` (cost 4) or as `LOAD`, `LOAD`, `ADDI` (cost 3), using an instruction with an immediate operand. Choosing well requires looking at whole subtrees, not single nodes, and the right choice differs from one processor to another.

Tree-pattern instruction selection treats each machine instruction as a pattern over IR trees and asks for the cheapest set of patterns that covers the whole tree. BURS solves this with dynamic programming bottom-up over the tree. BURG-style tools go further and precompute the dynamic programming into tables at build time, so selection at compile time is a table lookup per node. Because the instruction set lives entirely in the grammar, retargeting means writing a new grammar, not a new selector.

Beneficiaries and relevance. The project gives compiler-design students a small, complete and inspectable implementation of a real back-end technique: IR design, tree grammars, nonterminals and chain rules, dynamic programming, cover extraction, code emission and retargeting, each visible in the program's output. It also shows good engineering practice for compilers: a strict interface between machine-independent and machine-dependent code, and automatic semantic checking of generated code.

3. Problem Statement, Objectives, Scope and Limitations

3.1 Problem statement

Design and implement a BURS-style tree-pattern instruction selector that accepts an expression tree and emits target instructions for two simplified ISAs, keeping the IR and the matching engine common to both.

3.2 Objective-compliance matrix

#	Objective (approved in Review 1)	Measure of success	Final status	Evidence
1	Build an expression-tree representation for the selected IR inputs	All node types (six + DIV) built and printed; infix parser	Achieved	`src/ir/ir.h`, `ir.c`; IR tree in every run (App. B)
2	Define target-specific tree patterns, instruction templates and rule costs	Two grammars with costs and templates	Achieved	`targetA.gr` (11 rules), `targetB.gr` (17 rules); App. C
3	Implement bottom-up labeling / DP-style cost computation	Per-node best rule and cost for every nonterminal	Achieved	`burs_label()`, `burs_cost()`; cost table in App. B
4	Select a minimum-cost cover for the expression tree	Cheapest rule chosen where several match	Achieved	Rule-choice tests (Section 9); ADDI, SUBI, MULM, MULI
5	Backtrack the selected rules and emit target instructions	Code follows the cover in postorder	Achieved	`burs_print_cover()`, `burs_reduce()`
6	Support two simplified ISAs with a target-independent IR and engine	Same tree → two different, correct programs	Achieved	Retargeting summary in every run; Section 10
7	Prepare representative tests and reproducible evidence	Stored expected outputs; one-command regeneration	Achieved	`make test` (33 passed, 0 failed); `results/`

3.3 Scope

Expression-tree IR (CONST, VAR, ADD, SUB, MUL, DIV, LOAD); infix parser with `[e]` / `load(e)` for memory loads; BURG grammars for two targets; bottom-up labeling with rule costs and chain rules; cover printing, cost table and code emission; per-ISA simulators as correctness oracle; console driver with interactive, suite, exhaustive and benchmark modes; regression test suite.

3.4 Limitations (by design)

Expression trees only (no statements, control flow or DAG sharing); unlimited virtual registers on Target A (no register allocation or spilling); integer arithmetic only (division truncates toward zero; division by zero has no

value), without unary minus or remainder; costs are static per rule (no dynamic costs); educational ISAs, not a production processor.

4. Background / Related Work

4.1 Concepts used

Concept	Role in this project
Intermediate representation	Expression tree of `struct node` (op, children, value); the only data the engine and targets share.
Tree grammar	Each rule `nt: PATTERN = number (cost)` says "this subtree can be computed into nonterminal `nt` at this cost".
Nonterminal	Where a value lives: `reg` (in a register), `stk` (on the stack), `mem` (a memory operand), `imm` (an immediate).
Chain rule	Rule whose pattern is just another nonterminal, e.g. `stk: mem` = push a memory operand.
Dynamic programming	For every node and nonterminal, the cheapest rule given the children's best costs (optimal substructure on trees).
Cover	The set of selected rules, one per matched pattern, that together cover every node; its cost is the code cost.
BURS / BURG	Bottom-up rewrite system; BURG precomputes all reachable DP states into tables so labeling is a lookup per node.
Retargeting	Same IR and engine, different grammar → code for a different machine.

4.2 Comparison of approaches

Approach	Strength	Limitation	Used here?
Macro expansion / ad-hoc per node	Simple, one pass	One instruction per node; no immediates; poor retargetability	Used as the baseline in Section 10.3
Greedy maximal munch (top-down)	Fast, uses large patterns	Not optimal under general costs	No
Dynamic-programming tree matching (Aho–Ganapathi–Tjiang, iburg)	Optimal; costs computed at compile time	DP work at every node during compilation	Conceptually yes
BURS tables (BURG)	Optimal; constant work per node at compile time	Table generation offline; static costs	**Yes – the chosen approach**

4.3 Identified gap

Production code generators (LCC, LLVM's SelectionDAG matcher) are too large to study in a course project. The gap addressed here is a small, complete system in which the separation between one common BURS engine and two target descriptions is explicit, and in which every generated program is automatically verified.

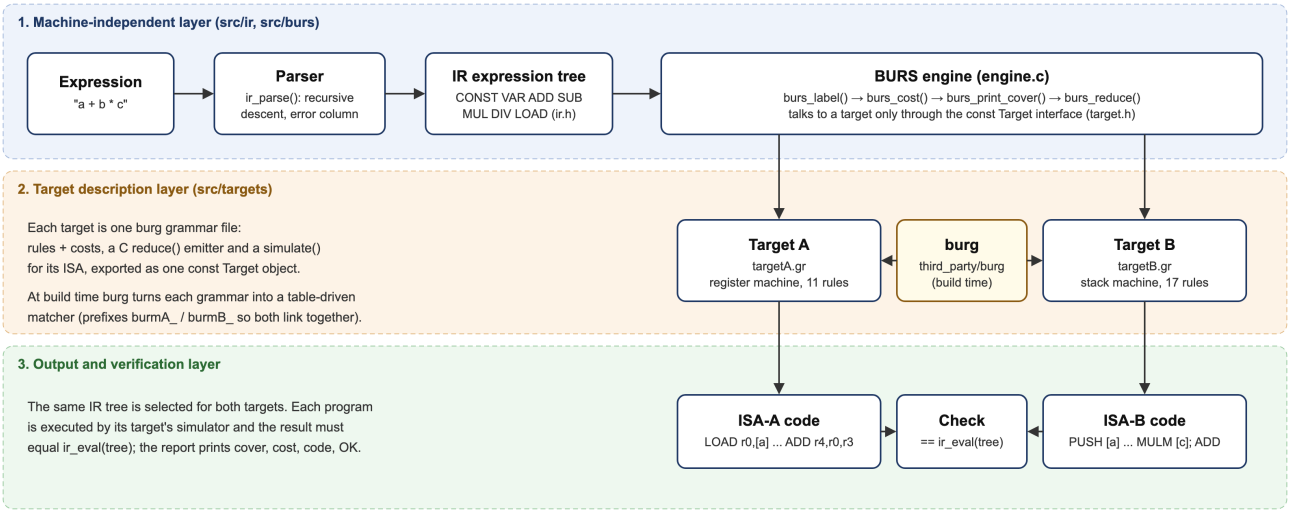
5. Requirements and Technology Stack

Category	Requirement / choice
Language	C99 (`-std=c99 -Wall -Wextra`); builds with zero warnings on Apple clang 21
Build	GNU make or BSD make; POSIX `sh` for test scripts
Generator	BURG (Fraser, Hanson, Proebsting; modernised by Chen Zhexuan), bundled in `third_party/burg`, built by the Makefile

Category	Requirement / choice
Libraries	C standard library only; no external dependencies
Hardware	Any laptop/desktop; tested on Apple M1, 8 GB RAM; memory use is a few MB
OS	macOS and Linux (any POSIX system with a C compiler)
Version control	Git; public GitHub repository
Grammars	`targetA.gr`, `targetB.gr` (App. C)
Test programs	`tests/cases.txt` (31 cases), built-in suite (11 expressions), exhaustive generator (7,140 trees), benchmark trees

6. System Design

6.1 Architecture and data flow



Final architecture. The machine-independent layer never names an instruction; each target is one grammar file; every program is checked against the IR value.

An input expression is parsed into the IR tree. The engine calls the selected target's generated `label` function, which walks the tree in postorder and stores a state number in every node. The engine then backtracks from the root with the target's start nonterminal, printing the cover and calling the target's `reduce()` for every selected rule, children first. The program is finally executed by the target's simulator and compared with `ir_eval()` of the tree. For two targets the same tree is simply relabeled with the second target's tables.

6.2 Modules and interfaces

Module	Files	Input → Output	Key functions
IR and front end	`src/ir/ir.h`, `ir.c`	text → IR tree; tree → value	`ir_parse`, `mk_*`, `printtree`, `ir_draw`, `ir_eval`, `freetree`
Target interface	`src/burs/target.h`	–	`struct Target`, `RuleInfo`, `EmitCtx`, `Operand`
BURS engine	`src/burs/engine.c`, `engine.h`	labeled tree → cost, cover, code	`burs_label`, `burs_cost`, `burs_print_cover`, `burs_print_cost_table`, `burs_reduce`
Target A description	`src/targets/targetA/targetA.gr`	rules → matcher; rule → instructions	burg rules, `rulesA[]`, `reduceA`, `simulateA`, `targetA`
Target B	`src/targets/targetB/targetB.gr`	rules → matcher; rule →	burg rules, `rulesB[]`, `reduceB`, `simulateB`, `targetB`

Module	Files	Input → Output	Key functions
description		instructions	
Driver / test harness	`src/main/main.c`, `tests/`	command line → report; cases → pass/fail	`process`, `run_suite`, `run_exhaustive`, `run_bench`, `run_tests.sh`

Review 2 interfaces mapped to code. `Node {type, children, value}` → `struct node {op, state_label, id, ival, name, left, right}`. `Rule {pattern, state, childStates, template, cost}` → a grammar rule plus its `RuleInfo` entry; child states come from the generated `burmX_nts[]`. `StateCost {state, cost, ruleUsed}` → `node->state_label`, queried by `rule(state, nt)` and `burs_cost()`. `MatchResult {rootState, selectedRules}` → the walk performed by `burs_print_cover()` and `burs_reduce()` from `(root, start_nt)`.

6.3 Key data structures

```
struct node {                   /* src/ir/ir.h – shared by every module          */
    int op;                     /* CONST=1 VAR=2 ADD=3 SUB=4 MUL=5 LOAD=6 DIV=7   */
    int state_label;            /* BURS state written by the target's _label()   */
    int id;                     /* postorder number n1..nN, used in all output   */
    long long ival;             /* CONST value                                   */
    char name[32];              /* VAR name                                      */
    struct node *left, *right;
};

typedef struct Target {         /* src/burs/target.h – everything the engine knows */
    const char *id, *name;
    int start_nt, num_nts;  const char *const *nt_names;
    int num_rules;          const RuleInfo *rules;        /* pattern, cost, asm   */
    int (*label)(NODEPTR_TYPE);                           /* burg-generated       */
    int (*rule)(int state, int goalnt);
    short **nts;
    NODEPTR_TYPE *(*kids)(NODEPTR_TYPE, int, NODEPTR_TYPE *);
    Operand (*reduce)(EmitCtx *, NODEPTR_TYPE, int rule, const Operand *kids);
    int (*simulate)(const EmitCtx *, const Operand *result, long long *value,
                    char *err, size_t errlen);
} Target;
```

6.4 The two target machines

Property	Target A	Target B
Machine model	3-address load/store, unlimited virtual registers	0-address stack machine
Nonterminals	`reg` (start), `imm`	`stk` (start), `mem`, `imm`
Rules	11	17
Instructions	`LOAD rd,[var]`, `LOAD rd,[rs]`, `LOADI`, `ADD`, `ADDI`, `SUB`, `SUBI`, `MUL`, `DIV`, `DIVI`	`PUSH [var]`, `PUSHI #k`, `LOAD`, `ADD/SUB/MUL/DIV`, `ADDM/SUBM/MULM/DIVM [var]`, `ADDI/SUBI/MULI/DIVI #k`
Special operand forms	immediate on the right of ADD/SUB/DIV	memory or immediate on the right of ADD/SUB/MUL/DIV
Cost model	1 per instruction, MUL = 2, DIV = 3	1 per instruction, MUL forms = 2, DIV forms = 3
Generated state rows	11	16

6.5 Design decisions

Decision	Alternatives considered	Reason
Use BURG tables rather than run-time DP	iburg-style DP at each node; hand-coded matcher	Linear-time labeling with tiny constant; optimality guaranteed by the generator; mirrors how BURS is taught.

Decision	Alternatives considered	Reason
Link both targets in one binary using `burg -p burmA` / `-p burmB`	One executable per target	The same tree can be selected for both targets in one run, which makes retargeting directly comparable.
All target knowledge behind one `const Target` object	`#ifdef` per target; global function names	The engine has no target-specific code; adding Target C needs one grammar, one Makefile rule and one array entry.
Emitter is a per-rule `reduce()` returning an operand	Template strings expanded by the engine	Operands differ by machine (`r3`, `#4`, `[a]`, top of stack); returning them keeps emission local to each rule.
Simulate every generated program	Compare with hand-written expected code only	An independent semantic oracle catches wrong covers and wrong emitters, and makes the exhaustive check possible.
Virtual registers, no allocation	Sethi–Ullman numbering, linear scan	Register allocation is outside the approved scope; virtual registers keep the cover-to-code mapping one-to-one.

7. Algorithms and Methodology

7.1 Parsing to IR

`ir_parse()` is a recursive-descent parser for `expr := term (('+'|'-') term)*`, `term := primary (('*'|'/') primary)*`, `primary := NUMBER | IDENT | '(' expr ')' | '[' expr ']' | load '(' expr ')'`. It yields left-associative trees with `*` and `/` binding tighter than +/-, numbers nodes in postorder (`n1`, `n2`, …) and reports the first error with its column. It runs in O(n) for n input characters.

7.2 Bottom-up labeling (dynamic programming)

For a node *v* and nonterminal *N*, the minimum cost is

```
cost(v, N) = min over rules r: N → pattern that match at v of
             [ cost(r) + Σ cost(leaf_i of pattern, nonterminal required at leaf_i) ]
```

followed by closure over chain rules (`N: M` adds `cost(r) + cost(v, M)`). Children are always solved before parents (postorder), so each subproblem is solved once. BURG performs this computation offline for every distinct situation that can arise, normalises costs relative to the cheapest nonterminal, and stores the result as a finite set of states plus transition tables indexed by operator and children's states. At compile time `burmX_label()` therefore does one table lookup per node: **O(n) time, O(n) space**. The grammars here have only 11 (A) and 16 (B) state rows, so the tables are a few hundred bytes.

Worked example (Target A, `load(a) + 4`), from `burs -v -t A "load(a) + 4"`:

Node	Nonterminal	Best rule	Cost	Why
n1 VAR a	reg	`reg: VAR`	1	only rule
n2 LOAD	reg	`reg: LOAD(reg)`	2	1 + cost(n1, reg)
n3 CONST 4	reg	`reg: CONST`	1	LOADI
n3 CONST 4	imm	`imm: CONST`	0	usable as an operand
n4 ADD	reg	`reg: ADD(reg,imm)`	3	1 + 2 + 0 = 3 beats `ADD(reg,reg)` = 1 + 2 + 1 = 4

7.3 Cover extraction and emission

Starting at (root, start nonterminal), `burs_reduce()` looks up the selected rule, asks the generated `kids()` function which subtrees the rule's nonterminal leaves correspond to (for `reg: ADD(reg,imm)` these are the left and right children; for the chain rule `stk: mem` it is the node itself), reduces each of them first, left to right, and then calls the target's `reduce()` with the children's operands. Code is therefore emitted in postorder, the order needed by both a register machine and a stack machine. Rules such as `imm: CONST` and `mem: VAR` emit nothing and return an operand (`#4`, `[a]`) that the parent folds into its instruction. Cover extraction and emission are O(n). `burs_cost()` recomputes the absolute cover cost by summing the selected rules' costs, which is used as a test oracle (cover cost = sum of rule costs).

7.4 Verification by simulation

Each target's `simulate()` parses the emitted text and executes it: Target A on a register file with definedness checks, Target B on an operand stack with underflow and final-depth checks. Variables take deterministic test values (a = 2, b = 3, …, overridable with `-D name=value`) and memory is modelled as `mem[x] = 1000 + x`. The simulated value must equal `ir_eval(tree)`. This independent oracle is what allows the exhaustive check of every small tree.

7.5 Complexity and trade-offs

Phase	Time	Space	Trade-off
Table generation (burg)	offline	tables	Done once per grammar at build time; grammar must have finitely many normalised states.
Parsing	O(n)	O(n)	–
Labeling	O(n)	O(1)/node	One lookup per node; no per-node cost arithmetic at compile time.
Cover + emission	O(n)	O(depth) stack	Recursive; depth of extremely deep unbalanced trees is bounded by the C stack.
Simulation	O(code)	O(regs)	Testing aid only.

Static costs make tables possible but rule out context-dependent costs (e.g. "constant fits in 8 bits"); iburg-style dynamic costs would remove that restriction at the price of DP work per node.

8. Implementation Details

8.1 Code organisation

Path	Lines	Contents
`src/ir/ir.h`, `ir.c`	507	Node type, builders, recursive-descent parser with error columns, printers, reference evaluator
`src/burs/target.h`, `engine.h`, `engine.c`	317	Target interface; label, cost, cover, cost-table, reduce; emitter and simulator helpers
`src/targets/targetA/targetA.gr`	194	11 rules, rule table, `reduceA`, `simulateA`, `const Target targetA`
`src/targets/targetB/targetB.gr`	208	17 rules, rule table, `reduceB`, `simulateB`, `const Target targetB`
`src/main/main.c`	575	Command-line driver: single expression, `-i`, `--suite`, `--exhaustive`, `--bench`, `--rules`
`tests/` and `Makefile`	194	Regression runner, results generator, 31 cases with expected outputs; three-stage build
`third_party/burg/`	–	BURG generator, unmodified, with its licence

8.2 Build process

`make` performs three stages: (1) compile the BURG generator from `third_party/burg`; (2) run `burg -p burmA -o build/gen/targetA.c targetA.gr` and the same with `burmB` for Target B, so both matchers have distinct symbol prefixes; (3) compile the IR, engine, driver and both generated files into `build/burs`. Each grammar's C footer (after the second `%%`) is copied by burg verbatim, which is where each target's rule table, emitter and simulator live.

8.3 Integration

The `%term` line `CONST=1 VAR=2 ADD=3 SUB=4 MUL=5 LOAD=6 DIV=7` is identical in both grammars and in `ir.h`; this is the single contract between the IR and all targets. `ir.h` defines `OP_LABEL`, `STATE_LABEL`, `LEFT_CHILD` and `RIGHT_CHILD`, which the generated matchers use to read the tree. The driver keeps an array `all_targets[] = { &targetA, &targetB }` and runs the same tree through each: label → cost table → cover → reduce → simulate → summary.

9. Testing Strategy and Test Cases

Every case in `tests/cases.txt` is run by `tests/run_tests.sh`; the complete output (stdout, stderr and exit status) must match the stored file in `tests/expected/` byte for byte. The suite and exhaustive modes are run as two further checks. Current result: **33 passed, 0 failed** (`results/test_report.txt`). "Value" is the reference result of `ir_eval()` with a = 2, b = 3, c = 4, … and `mem[x] = 1000 + x`; "OK" means the simulated program produced that value. Case IDs are prefixed by type: `pos` positive, `bnd` boundary, `rule` rule choice, `neg` negative (exit 2), `fail` valid input whose value is undefined (exit 3).

ID	Input (options)	Expected	Actual output	Result
pos_a_plus_b_mul_c	a + b * c (-v)	Valid tree, min-cost cover, code for both targets; value 14	A: LOAD r0,[a]; LOAD r1,[b]; LOAD r2,[c]; MUL r3,r1,r2; ADD r4,r0,r3 (cost 6) OK ; B: PUSH [a]; PUSH [b]; MULM [c]; ADD (cost 5) OK	PASS
pos_paren_mul	(a + b) * c (-v)	Valid tree, min-cost cover, code for both targets; value 20	A: LOAD r0,[a]; LOAD r1,[b]; ADD r2,r0,r1; LOAD r3,[c]; MUL r4,r2,r3 (cost 6) OK ; B: PUSH [a]; ADDM [b]; MULM [c] (cost 4) OK	PASS
pos_a_plus_b_plus_c	a + b + c	Valid tree, min-cost cover, code for both targets; value 9	A: LOAD r0,[a]; LOAD r1,[b]; ADD r2,r0,r1; LOAD r3,[c]; ADD r4,r2,r3 (cost 5) OK ; B: PUSH [a]; ADDM [b]; ADDM [c] (cost 3) OK	PASS
pos_a_mul_paren	a * (b + c)	Valid tree, min-cost cover, code for both targets; value 14	A: LOAD r0,[a]; LOAD r1,[b]; LOAD r2,[c]; ADD r3,r1,r2; MUL r4,r0,r3 (cost 6) OK ; B: PUSH [a]; PUSH [b]; ADDM [c]; MUL (cost 5) OK	PASS
pos_a_minus_b_plus_c	a - b + c	Valid tree, min-cost cover, code for both	A: LOAD r0,[a]; LOAD r1,[b]; SUB r2,r0,r1; LOAD	PASS

ID	Input (options)	Expected	Actual output	Result
		targets; value 3	r3,[c]; ADD r4,r2,r3 (cost 5) OK ; B: PUSH [a]; SUBM [b]; ADDM [c] (cost 3) OK	
pos_load_plus_const	load(a) + 4 (-v)	Valid tree, min-cost cover, code for both targets; value 1006	A: LOAD r0,[a]; LOAD r1,[r0]; ADDI r2,r1,#4 (cost 3) OK ; B: PUSH [a]; LOAD; ADDI #4 (cost 3) OK	PASS
pos_division	(a + b) / c	Valid tree, min-cost cover, code for both targets; value 1	A: LOAD r0,[a]; LOAD r1,[b]; ADD r2,r0,r1; LOAD r3,[c]; DIV r4,r2,r3 (cost 7) OK ; B: PUSH [a]; ADDM [b]; DIVM [c] (cost 5) OK	PASS
pos_division_mixed	((a*b)+((a-c)*d/b)+a*2)	Valid tree, min-cost cover, code for both targets; value 7	A: 15 instructions: LOAD r0,[a]; LOAD r1,[b] … ADD r14,r10,r13 (cost 20) OK ; B: 10 instructions: PUSH [a]; MULM [b] … ADD (cost 15) OK	PASS
bnd_single_var	a	One load / push, no extra rules; value 2	A: LOAD r0,[a] (cost 1) OK ; B: PUSH [a] (cost 1) OK	PASS
bnd_single_const	42	One LOADI / PUSHI; value 42	A: LOADI r0,#42 (cost 1) OK ; B: PUSHI #42 (cost 1) OK	PASS
bnd_deep_nesting	a+b*(c-(d+e*(f-(g+h*(i-(j+k*(l-(m+n*(o-(p+q*(r-(s+t))))))))))))	20 variables, 19 levels; correct postorder; value 11519999	A: 39 instructions: LOAD r0,[a]; LOAD r1,[b] … ADD r38,r0,r37 (cost 45) OK ; B: 38 instructions: PUSH [a]; PUSH [b] … ADD (cost 44) OK	PASS
bnd_named_vars	base + count * 8 (-D count=10 -D base=100)	Multi-letter names; values set with -D; value 180	A: LOAD r0,[base]; LOAD r1,[count]; LOADI r2,#8; MUL r3,r1,r2; ADD r4,r0,r3 (cost 6) OK ; B: PUSH [base]; PUSH [count]; MULI #8; ADD (cost 5) OK	PASS
bnd_nested_loads	[[a] + 1]	LOAD of LOAD; ADDI inside the address; value 2003	A: LOAD r0,[a]; LOAD r1,[r0]; ADDI r2,r1,#1; LOAD r3,[r2] (cost 4) OK ; B: PUSH [a]; LOAD; ADDI #1; LOAD (cost 4) OK	PASS
bnd_large_const	a * 1000000 - 999999	Large immediates folded; no overflow; value 1000001	A: LOAD r0,[a]; LOADI r1,#1000000; MUL r2,r0,r1; SUBI r3,r2,#999999 (cost 5) OK ; B: PUSH [a]; MULI #1000000; SUBI #999999 (cost 4) OK	PASS

ID	Input (options)	Expected	Actual output	Result
bnd_div_truncates	(0 - 7) / 2	Negative quotient truncates toward zero; value -3	A: LOADI r0,#0; SUBI r1,r0,#7; DIVI r2,r1,#2 (cost 5) OK ; B: PUSHI #0; SUBI #7; DIVI #2 (cost 5) OK	PASS
rule_addi_vs_loadi	a + 7 (-t A)	ADDI chosen: cost 2, not LOADI + ADD (3)	A: LOAD r0,[a]; ADDI r1,r0,#7 (cost 2) OK	PASS
rule_const_on_left	7 + a (-t A)	No commutative rule, so LOADI + ADD (cost 3)	A: LOADI r0,#7; LOAD r1,[a]; ADD r2,r0,r1 (cost 3) OK	PASS
rule_subi	a - 1 (-t A)	SUBI chosen: cost 2, not LOADI + SUB (3)	A: LOAD r0,[a]; SUBI r1,r0,#1 (cost 2) OK	PASS
rule_mem_operand	a * b (-t B)	MULM chosen: cost 3, not PUSH + PUSH + MUL (4)	B: PUSH [a]; MULM [b] (cost 3) OK	PASS
rule_muli	a * 5 (-t B)	MULI chosen: cost 3, not PUSHI + MUL (4)	B: PUSH [a]; MULI #5 (cost 3) OK	PASS
rule_divi	a / 2 (-t A)	DIVI chosen: cost 4, not LOADI + DIV (5)	A: LOAD r0,[a]; DIVI r1,r0,#2 (cost 4) OK	PASS
rule_divm	a / b (-t B)	DIVM chosen: cost 4, not PUSH + PUSH + DIV (5)	B: PUSH [a]; DIVM [b] (cost 4) OK	PASS
neg_modulo	a % b	Rejected with a clear message, exit 2	exit 2: column 3: operator '%' is not supported (IR has ADD, SUB, MUL, DIV, LOAD only)	PASS
neg_missing_operand	a + * b	Rejected with a clear message, exit 2	exit 2: column 5: expected a number, variable, '(' or '[' but found '*'	PASS
neg_unbalanced_paren	(a + b	Rejected with a clear message, exit 2	exit 2: column 7: expected ')' but found end of input	PASS
neg_unbalanced_bracket	[a + b	Rejected with a clear message, exit 2	exit 2: column 7: expected ']' but found end of input	PASS
neg_trailing_token	a b	Rejected with a clear message, exit 2	exit 2: column 3: unexpected 'b' after a complete expression	PASS
neg_unary_minus	a * -b	Rejected with a clear message, exit 2	exit 2: column 5: unary minus is not supported; write (0 - x)	PASS
neg_empty	(empty)	Rejected with a clear message, exit 2	exit 2: column 1: empty expression	PASS
neg_bad_char	a + $	Rejected with a clear message, exit 2	exit 2: column 5: expected a number, variable, '(' or '[' but found '$'	PASS
fail_div_by_zero	a / (b - b)	Code generated; value undefined (divide by zero); check not run, exit 3	A: LOAD r0,[a]; LOAD r1,[b]; LOAD r2,[b]; SUB r3,r1,r2; DIV r4,r0,r3 (cost 7) check not run ; B: PUSH [a]; PUSH [b]; SUBM [b];	PASS

ID	Input (options)	Expected	Actual output	Result
			`DIV` (cost 6) check not run ; exit 3	
`--suite` (regression)	11 review expressions	All PASS	11/11 expressions passed	PASS
`--exhaustive` (stress)	7,140 trees (6,846 checked) x 2 targets	0 failures	13,692 target runs, 0 failures	PASS

Test oracles used (Review 1 §12.1): the printed tree matches the input's operator structure; the reported cover cost equals the sum of the selected rules' costs; emitted code follows the selected cover; the simulated value equals the IR value; and outputs are stored with the test cases for reproducibility.

10. Results and Evaluation

10.1 Review sample expressions on both targets

Expression	Target A code	A cost	Target B code	B cost	Value	Simulated
`a + b * c`	`LOAD r0,[a]; LOAD r1,[b]; LOAD r2,[c]; MUL r3,r1,r2; ADD r4,r0,r3`	6	`PUSH [a]; PUSH [b]; MULM [c]; ADD`	5	14	A ✓ B ✓
`(a + b) * c`	`LOAD r0,[a]; LOAD r1,[b]; ADD r2,r0,r1; LOAD r3,[c]; MUL r4,r2,r3`	6	`PUSH [a]; ADDM [b]; MULM [c]`	4	20	A ✓ B ✓
`a + b + c`	`LOAD r0,[a]; LOAD r1,[b]; ADD r2,r0,r1; LOAD r3,[c]; ADD r4,r2,r3`	5	`PUSH [a]; ADDM [b]; ADDM [c]`	3	9	A ✓ B ✓
`a * (b + c)`	`LOAD r0,[a]; LOAD r1,[b]; LOAD r2,[c]; ADD r3,r1,r2; MUL r4,r0,r3`	6	`PUSH [a]; PUSH [b]; ADDM [c]; MUL`	5	14	A ✓ B ✓
`a - b + c`	`LOAD r0,[a]; LOAD r1,[b]; SUB r2,r0,r1; LOAD r3,[c]; ADD r4,r2,r3`	5	`PUSH [a]; SUBM [b]; ADDM [c]`	3	3	A ✓ B ✓
`load(a) + 4`	`LOAD r0,[a]; LOAD r1,[r0]; ADDI r2,r1,#4`	3	`PUSH [a]; LOAD; ADDI #4`	3	1006	A ✓ B ✓
`(a + b) / c`	`LOAD r0,[a]; LOAD r1,[b]; ADD r2,r0,r1; LOAD r3,[c]; DIV r4,r2,r3`	7	`PUSH [a]; ADDM [b]; DIVM [c]`	5	1	A ✓ B ✓

Target A's code and costs are identical to the values recorded in the Review 2 document (Section 13), confirming that the restructured engine preserves the validated behaviour. Target B, pending at Review 2, is complete.

10.2 Review suite summary (`burs --suite`)

#	Expression	A instructions	A cost	B instructions	B cost	Check
1	`a + b * c`	5	6	4	5	PASS
2	`(a + b) * c`	5	6	3	4	PASS
3	`load(a) + 4`	3	3	3	3	PASS
4	`a - b + c`	5	5	3	3	PASS
5	`a + b + c`	5	5	3	3	PASS
6	`a * (b + c)`	5	6	4	5	PASS
7	`a`	1	1	1	1	PASS
8	`7`	1	1	1	1	PASS
9	`a * 3 - 1`	4	5	3	4	PASS
10	`[a + 8] * [b]`	6	7	6	7	PASS

#	Expression	A instructions	A cost	B instructions	B cost	Check
11	`(a + b) / c`	5	7	3	5	PASS

Result: **11/11 expressions passed** (`results/suite_summary.txt`).

10.3 Exhaustive correctness check and comparison with a naive selector

`burs --exhaustive` builds every tree of depth ≤ 2 over the leaves {a, b, 3} with ADD, SUB, MUL, DIV and LOAD (7,140 trees). The 294 trees that divide by zero with the test values (such as `a / (b - b)`) have no reference value and are skipped; the other 6,846 run through both targets. The baseline is a naive selector that emits one instruction per IR node (LOAD/LOADI or PUSH/PUSHI for leaves, the plain operator for interior nodes) under the same cost model; it never uses immediate or memory operand forms.

Selector	Average cover cost	Reduction vs naive	Cheaper than naive on	Failures	Max cover cost
Naive, one instruction per node	8.49	–	–	–	–
BURS Target A	8.05	5.1%	2655 of 6846 trees	0	13
BURS Target B	6.72	20.9%	6762 of 6846 trees	0	11

13692 target runs in total,, 0 failures (`results/exhaustive_check.txt`).

Interpretation. All 13,692 programs are correct. BURS never costs more than the naive baseline (the baseline cover is always available to the matcher) and is strictly cheaper wherever a special operand form applies. Target B benefits far more (20.9%) because whenever the right operand of a binary node is a variable or a constant, `ADDM`/`MULM`/`DIVM`/`ADDI` absorb it and a `PUSH` disappears; this applies to 6,762 of the 6,846 trees. Target A only gains when a constant appears on the right of ADD, SUB or DIV, since it has no memory-operand arithmetic and no commutative rules (`7 + a` is not folded; see Section 11).

10.4 Scaling benchmark (`burs --bench`)

Random trees of depth 8 to 18 (405 to 239,710 nodes); time for the labeling pass plus the cover-cost backtrack, best of five runs on an Apple M1.

Tree depth	Nodes	A: time (ms)	A: ns / node	A: cover cost	B: time (ms)	B: ns / node	B: cover cost
8	405	0.004	9.9	551	0.005	12.3	478
10	1587	0.018	11.3	2168	0.021	13.2	1900
12	4986	0.071	14.2	6765	0.096	19.3	5953
14	13229	0.200	15.1	17940	0.232	17.5	15735
16	59790	0.893	14.9	80804	1.081	18.1	70894
18	239710	4.152	17.3	323609	5.058	21.1	283646

Source: `results/benchmark.txt` (timings vary by machine).

Interpretation. Time per node stays within roughly 9–21 ns while the tree grows by a factor of more than 500, i.e. labeling is linear in tree size as predicted in Section 7.5. The mild growth in ns/node comes from cache misses on larger trees, not from the algorithm. Even a 240,000-node expression is labeled in about 5 ms.

11. Error Handling, Limitations and Known Issues

11.1 Error handling

Condition	Detection	Behaviour	Exit status
Malformed expression (missing operand, unbalanced `(` or `[`, trailing token, bad character, empty)	Parser	`ERROR: invalid expression, column N: …` with the expected and found token	2
Unsupported operator (`%`, `^`, `&`, bitwise or) or unary minus	Parser	Message names the operator and the supported set, or suggests `(0 - x)`	2
Division by zero with the current test values	`ir_eval()` / `ir_div()`	Code is generated; reference shown as undefined; simulation not run (change values with `-D`)	3
Integer constant overflow / identifier > 31 chars	Parser	Clear error message	2
Tree not derivable from the start nonterminal	`burs_label()` returns 0	`no cover` error (cannot occur with the current grammars; kept for new targets)	3
Generated code wrong	Simulator	`MISMATCH` or a precise simulator error (undefined register, stack underflow)	3
More than 512 instructions	`EmitCtx` overflow flag	Error instead of buffer overrun	3
Bad command line (unknown option, missing argument, unknown target)	Driver	Usage message	1

11.2 Limitations and their impact

Limitation	Impact	Possible remedy
Expression trees only (no DAGs)	Common subexpressions are recomputed	DAG splitting before selection
No register allocation on Target A	One virtual register per value; deep trees use many (39 for the deep-nesting test)	Sethi–Ullman numbering or linear-scan allocation
No commutative rules	`7 + a` uses `LOADI` + `ADD` (cost 3) instead of `ADDI` (cost 2)	Add `reg: ADD(imm,reg)` and `stk: ADD(mem,stk)` rules
Static costs	Cannot express “small immediates are cheaper”	Dynamic costs (iburg style)
Cost ties fixed at table-generation time	Among equal-cost covers, the generator’s choice is used	Acceptable: all tied covers have the same cost
No remainder, unary minus, comparisons	Such inputs are rejected	Add IR operators and rules (as was done for DIV)
Deep recursion in engine and parser	Bounded by the C stack for extremely deep unbalanced trees	Iterative traversal

11.3 Defect log (final)

ID	Description	Module	Severity	Status
D1	NT index ambiguity: emitter called with NT 3 instead of `reg` = 1 (Review 2)	Emitter	Med	Fixed – NT numbers now taken from generated `burmX_*_NT` macros
D2	`kids()` child ordering misread for LOAD (Review 2)	BURS matcher	Low	Fixed
D3	Target-B emitter not written (Review 2)	Target B	Med	Fixed – `reduceB()`, `simulateB()`

ID	Description	Module	Severity	Status
D4	`-t A` still ran both targets: the "found" flag leaked into the match test	Driver	Med	Fixed; covered by `-t` regression cases
D5	Deep-nesting test case had one extra `)`; `-a` was read as an option	Tests	Low	Fixed; cases corrected and expected outputs regenerated
D6	Exhaustive summary reported "all covered" regardless of failures	Driver	Low	Fixed – per-target failure counts printed
D7	Division was rejected (`a / b` → exit 2); requested during preparation	IR / targets	Med	Fixed – DIV added to IR, parser, both grammars and simulators; 6 new tests

12. Member-Wise Contribution and Final Contribution Matrix

The ownership below follows the responsibility split approved in Review 1 and recorded in Review 2, mapped to the final source files. **Per-member commit links are to be added from the repository history before submission.**

12.1 Mandatory final contribution matrix

Member	Primary technical ownership	Final deliverable produced	Evidence / commit / test	Integration / support contribution	Viva module
Member 1 – Vineet Kumar Mishra (Project Lead / System Integrator)	IR module and front end; overall architecture	`ir.h` node contract and `%term` numbering; `ir.c` builders, parser, printers, `ir_eval`	`src/ir/`; negative tests `neg_*`; IR tree output; **commits**	Shared IR contract used by both grammars; build pipeline; objective-compliance matrix	Architecture, IR design, parser and error handling
Member 2 – Mayank Verma (Target Description Owner)	Target rule tables and cost models for A and B	11-rule `targetA.gr` and 17-rule `targetB.gr` rule sections and `RuleInfo` tables	`src/targets/*/`; rule-choice tests `rule_*`; `results/rule_tables.txt`; **commits**	Nonterminal design (`reg`, `stk`, `mem`, `imm`) used by the engine; references	Tree grammars, chain rules, costs, why ADDI/MULM win
Member 3 – Utsav Keshrwani (Core BURS Engine Owner)	BURS matching / cost engine and burg integration	`engine.c` (`burs_label`, `burs_cost`, cover, cost table, `burs_reduce`); `target.h`; prefixed generation	`src/burs/`; cost tables in `-v` runs; `--exhaustive`, `--bench`; **commits**	Target interface consumed by both targets and the driver	Dynamic programming, BURG tables, complexity
Member 4 – Ahmad Azim (Testing / Evaluation / Release Owner)	Emitters, simulators, driver, test suite, repository	`reduceA/B`, `simulateA/B`, `main.c`, `tests/`, `results/`, README, GitHub repository	`tests/run_tests.sh` (33/33), `results/`, repository `main` branch; **commits**	End-to-end integration, evaluation (baseline, benchmark), release	Emission, verification, test suite, reproducibility

12.2 Presentation and demonstration allocation

Presenter	Time	Content	Live command(s)
Member 1 – Vineet	4 min	Problem, objectives, Review 2 closure, architecture, IR and parser	`make clean && make; ./build/burs "a % b"`
Member 2 – Mayank	4–5 min	Both grammars, nonterminals, chain rules, costs; rule choice	`./build/burs --rules; ./build/burs -t A "a + 7"`
Member 3 – Utsav	4–5 min	Labeling, cost table, cover backtracking, complexity, benchmark	`./build/burs -v -t A "load(a) + 4"; ./build/burs --bench`
Member 4 – Ahmad	4–5 min	Emission on both ISAs, simulation check, test suite, results, repository, limitations	`./build/burs "a + b * c"; make test; ./build/burs --exhaustive`
All members	8–12 min	Evaluator-selected inputs, source-code locations, design choices, extensions	`./build/burs -i`

13. Conclusion and Future Work

Objectives achieved. All seven approved objectives are met (Section 3.2). One IR and one BURS engine produce different, cost-optimal and automatically verified code for a register machine and a stack machine; the only target-specific code is one grammar file per machine. Target A reproduces the Review 2 results exactly; Target B is complete. The system is reproducible with `make && make test`, passes 33/33 checks including 13,692 exhaustive runs, lowers code cost by 5.1% (A) and 20.9% (B) against a naive selector, and labels in linear time. Adding division later required one IR operator and one rule group per grammar, with no change to the engine, which is the retargetability claim in practice.

Key learning. (1) Optimal instruction selection on trees is a clean dynamic-programming problem, and BURG moves all of that work to build time. (2) Nonterminals model *where a value lives*, and chain rules let one node offer several forms to its parent; this is what makes memory and immediate operands come out naturally. (3) A strict interface between machine-independent and machine-dependent code is what makes retargeting cheap. (4) An executable oracle (simulation) is far stronger evidence than comparing against expected listings by eye.

Future work. Register allocation (Sethi–Ullman labeling or linear scan) for Target A; DAG and statement support; commutative and addressing-mode rules (e.g. `LOAD rd,[rs+#k]`); dynamic costs; a third target modelled on a real ISA subset such as RISC-V RV32I; a small visualiser that draws the cover over the tree.

Publication / extension potential. The project is suitable as an educational case study or lab module: a compact, fully reproducible demonstration that one matching engine retargets to two ISAs, with automatic semantic verification and quantitative comparison. A short paper or lab handout would need the planned third target and a register-allocation comparison to go beyond what is already in the literature.

14. References and Acknowledgements

1. A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2006, Chapter 8 (Code Generation).
2. A. V. Aho, M. Ganapathi and S. W. K. Tjiang, "Code Generation Using Tree Matching and Dynamic Programming," *ACM Transactions on Programming Languages and Systems*, vol. 11, no. 4, pp. 491–516, 1989.
3. C. W. Fraser, D. R. Hanson and T. A. Proebsting, "Engineering a Simple, Efficient Code-Generator Generator," *ACM Letters on Programming Languages and Systems*, vol. 1, no. 3, pp. 213–226, 1992.
4. C. W. Fraser, R. R. Henry and T. A. Proebsting, "BURG – Fast Optimal Instruction Selection and Tree Parsing," *ACM SIGPLAN Notices*, vol. 27, no. 4, pp. 68–76, 1992.
5. T. A. Proebsting, "Simple and Efficient BURS Table Generation," *Proc. ACM SIGPLAN PLDI*, 1992, pp. 331–340.
6. Chen Zhexuan (CloseToZero), *burg* – modernised C implementation of BURG, https://github.com/CloseToZero/burg.
7. Course notes and lecture material, BCSE307L Compiler Design: back end, instruction selection and intermediate representations.

Acknowledgement of reused resources. `third_party/burg` is the BURG generator by Fraser, Hanson and Proebsting in the modernised version by Chen Zhexuan; it is included unmodified as a build tool under its own licence (`COPYRIGHT`, `LICENSE.TXT`), and no BURG source is part of the team's code. Development of the final implementation and of this report was assisted by an AI coding assistant (Claude, by Anthropic); the repository commit records this assistance. The team reviewed, tested and is responsible for all submitted work.

Appendix A – User Manual and Installation Guide

Prerequisites: a C compiler (`cc`, `gcc` or `clang`), `make`, and `git` to clone.

```
git clone https://github.com/ahmadazim542/burs-instruction-selector.git
cd burs-instruction-selector
make                    # builds burg, generates both matchers, builds build/burs
make test               # 33 passed, 0 failed
```

Command	Purpose
`./build/burs "a + b * c"`	IR tree, cover, code, cost and simulation check for both targets
`./build/burs -v "load(a) + 4"`	Adds the per-node BURS cost table
`./build/burs -t A "a - 1"`	One target only (`-t A`, `-t B`, `-t both`)
`./build/burs -D a=10 "a * a"`	Set variable values used by the simulation check
`./build/burs --rules`	Rule tables of both targets
`./build/burs --suite`	Summary table for the review sample expressions
`./build/burs --exhaustive`	All 7,140 trees of depth ≤ 2 on both targets, with baseline comparison
`./build/burs --bench`	Scaling benchmark
`./build/burs -i`	Interactive mode; type `quit` to leave
`make results`	Regenerate the evidence files in `results/`

Expression syntax: `+ - * /` (division truncates toward zero), non-negative integer constants, identifiers, parentheses, and memory loads `[e]` or `load(e)`. Exit status: 0 success, 1 usage error, 2 invalid expression, 3 selection or verification failure.

Appendix B – Sample Runs

B.1 `./build/burs "a + b * c"` (both targets, complete output)

```
Expression   : a + b * c
IR (prefix)  : ADD(VAR a, MUL(VAR b, VAR c))
IR tree      :
    ADD   [n5]
    |-- VAR a   [n1]
    `-- MUL   [n4]
        |-- VAR b   [n2]
        `-- VAR c   [n3]
Test values  : a=2 b=3 c=4, mem[x]=1000+x
Reference    : 14

========================================================================
Target A : 3-address load/store register machine (LOAD/LOADI/ADD/ADDI/SUB/SUBI/MUL/DIV/DIVI)
========================================================================
Selected cover (backtracked from root, goal 'reg'):
    reg: ADD(reg,reg)                     n5    cost 1   ADD   rd, ra, rb
    . reg: VAR                            n1    cost 1   LOAD  rd, [var]
    . reg: MUL(reg,reg)                   n4    cost 2   MUL   rd, ra, rb
    . . reg: VAR                          n2    cost 1   LOAD  rd, [var]
    . . reg: VAR                          n3    cost 1   LOAD  rd, [var]

Generated code:
      1  LOAD r0,[a]
      2  LOAD r1,[b]
      3  LOAD r2,[c]
      4  MUL r3,r1,r2
      5  ADD r4,r0,r3

Total cover cost : 6   (5 instructions)
Simulation       : result 14, reference 14   OK
```

```
========================================================================
Target B : zero-address stack machine (PUSH/PUSHI/LOAD/ADD/ADDM/ADDI/SUB.../MUL.../DIV...)
========================================================================
Selected cover (backtracked from root, goal 'stk'):
    stk: ADD(stk,stk)                       n5    cost 1   ADD
    . stk: mem                              n1    cost 1   PUSH  [var]
    . . mem: VAR                            n1    cost 0   (operand [var], no code)
    . stk: MUL(stk,mem)                     n4    cost 2   MULM  [var]
    . . stk: mem                            n2    cost 1   PUSH  [var]
    . . . mem: VAR                          n2    cost 0   (operand [var], no code)
    . . mem: VAR                            n3    cost 0   (operand [var], no code)

Generated code:
      1  PUSH [a]
      2  PUSH [b]
      3  MULM [c]
      4  ADD

Total cover cost : 5   (4 instructions)
Simulation       : result 14, reference 14   OK

------------------------------------------------------------------------
Retargeting summary (same IR tree, same BURS engine):
    target   instrs   cost   check
    A        5        6      OK
    B        4        5      OK
------------------------------------------------------------------------
```

B.2 `./build/burs -v -t A "load(a) + 4"` (with the BURS cost table)

```
Expression   : load(a) + 4
IR (prefix)  : ADD(LOAD(VAR a), CONST 4)
IR tree      :
    ADD   [n4]
    |-- LOAD   [n2]
    |   `-- VAR a   [n1]
    `-- CONST 4   [n3]
Test values  : a=2, mem[x]=1000+x
Reference    : 1006

========================================================================
Target A : 3-address load/store register machine (LOAD/LOADI/ADD/ADDI/SUB/SUBI/MUL/DIV/DIVI)
========================================================================
BURS cost table (bottom-up labeling, best rule per nonterminal):
    node               NT    best rule              cost
    ----               --    ---------              ----
    n1 VAR a           reg   reg: VAR                  1
    n2 LOAD            reg   reg: LOAD(reg)            2
    n3 CONST 4         reg   reg: CONST                1
                       imm   imm: CONST                0
    n4 ADD             reg   reg: ADD(reg,imm)         3

Selected cover (backtracked from root, goal 'reg'):
    reg: ADD(reg,imm)                       n4    cost 1   ADDI  rd, ra, #k
    . reg: LOAD(reg)                        n2    cost 1   LOAD  rd, [rs]
    . . reg: VAR                            n1    cost 1   LOAD  rd, [var]
    . imm: CONST                            n3    cost 0   (operand #k, no code)

Generated code:
      1  LOAD r0,[a]
      2  LOAD r1,[r0]
      3  ADDI r2,r1,#4

Total cover cost : 3   (3 instructions)
Simulation       : result 1006, reference 1006   OK
```

Appendix C – Grammar Specification

Both grammars use `%term CONST=1 VAR=2 ADD=3 SUB=4 MUL=5 LOAD=6 DIV=7`. Each rule reads *nonterminal: pattern = rule number (cost)*.

Target A (`src/targets/targetA/targetA.gr`) – start nonterminal `reg`

```
reg:    VAR     = 1 (1);
reg:    CONST       = 2 (1);
imm:    CONST       = 3 (0);
reg:    LOAD(reg)   = 4 (1);
reg:    ADD(reg,reg)    = 5 (1);
reg:    ADD(reg,imm)    = 6 (1);
reg:    SUB(reg,reg)    = 7 (1);
reg:    SUB(reg,imm)    = 8 (1);
reg:    MUL(reg,reg)    = 9 (2);
reg:    DIV(reg,reg)    = 10 (3);
reg:    DIV(reg,imm)    = 11 (3);
```

Target B (`src/targets/targetB/targetB.gr`) – start nonterminal `stk`

```
stk:    mem     = 1 (1);
stk:    imm     = 2 (1);
mem:    VAR     = 3 (0);
imm:    CONST       = 4 (0);
stk:    LOAD(stk)   = 5 (1);
stk:    ADD(stk,stk)    = 6 (1);
stk:    ADD(stk,mem)    = 7 (1);
stk:    ADD(stk,imm)    = 8 (1);
stk:    SUB(stk,stk)    = 9 (1);
stk:    SUB(stk,mem)    = 10 (1);
stk:    SUB(stk,imm)    = 11 (1);
stk:    MUL(stk,stk)    = 12 (2);
stk:    MUL(stk,mem)    = 13 (2);
stk:    MUL(stk,imm)    = 14 (2);
stk:    DIV(stk,stk)    = 15 (3);
stk:    DIV(stk,mem)    = 16 (3);
stk:    DIV(stk,imm)    = 17 (3);
```

Rule tables with instruction templates, as printed by `./build/burs --rules`:

```
Target A: 3-address load/store register machine (LOAD/LOADI/ADD/ADDI/SUB/SUBI/MUL/DIV/DIVI)
    #   rule (pattern)          cost instruction template
    --  --------------          ---- --------------------
    1   reg: VAR                   1 LOAD  rd, [var]
    2   reg: CONST                 1 LOADI rd, #k
    3   imm: CONST                 0 (operand #k, no code)
    4   reg: LOAD(reg)             1 LOAD  rd, [rs]
    5   reg: ADD(reg,reg)          1 ADD   rd, ra, rb
    6   reg: ADD(reg,imm)          1 ADDI  rd, ra, #k
    7   reg: SUB(reg,reg)          1 SUB   rd, ra, rb
    8   reg: SUB(reg,imm)          1 SUBI  rd, ra, #k
    9   reg: MUL(reg,reg)          2 MUL   rd, ra, rb
    10  reg: DIV(reg,reg)          3 DIV   rd, ra, rb
    11  reg: DIV(reg,imm)          3 DIVI  rd, ra, #k

Target B: zero-address stack machine (PUSH/PUSHI/LOAD/ADD/ADDM/ADDI/SUB.../MUL.../DIV...)
    #   rule (pattern)          cost instruction template
    --  --------------          ---- --------------------
    1   stk: mem                   1 PUSH  [var]
    2   stk: imm                   1 PUSHI #k
    3   mem: VAR                   0 (operand [var], no code)
    4   imm: CONST                 0 (operand #k, no code)
    5   stk: LOAD(stk)             1 LOAD
    6   stk: ADD(stk,stk)          1 ADD
    7   stk: ADD(stk,mem)          1 ADDM  [var]
    8   stk: ADD(stk,imm)          1 ADDI  #k
    9   stk: SUB(stk,stk)          1 SUB
    10  stk: SUB(stk,mem)          1 SUBM  [var]
```

```
    11  stk: SUB(stk,imm)           1 SUBI  #k
    12  stk: MUL(stk,stk)           2 MUL
    13  stk: MUL(stk,mem)           2 MULM  [var]
    14  stk: MUL(stk,imm)           2 MULI  #k
    15  stk: DIV(stk,stk)           3 DIV
    16  stk: DIV(stk,mem)           3 DIVM  [var]
    17  stk: DIV(stk,imm)           3 DIVI  #k
```

Appendix D – Review 3 Demonstration Plan, Checklist and Records

D.1 Final demonstration sequence

Stage	Demonstration activity	Command	Expected evidence
1. Environment	Show repository, dependencies, build	`git log --oneline`; `make clean && make`	Three build stages, zero warnings
2. Normal use case	Representative valid input through the whole system	`./build/burs "a + b * c"`	Tree, two covers, two programs, both `OK`
3. Alternative / boundary	Leaf, deep nesting, nested loads, named variables	`./build/burs 42`; `./build/burs "[[a] + 1]"`	Correct leaf handling; `OK`
4. Invalid / failure	Malformed and unsupported input	`./build/burs "(a + b"`; `./build/burs "a % b"`	Error with column, exit status 2
5. Internal evidence	DP cost table, rule tables, source code	`./build/burs -v -t A "load(a) + 4"`; open `engine.c`	Per-node best rule and cost; ADDI chosen over LOADI + ADD
6. Evaluation result	Test summary, exhaustive check, benchmark	`make test`; `./build/burs --exhaustive`; `./build/burs --bench`	33 passed; 0 failures; 5.1% / 20.9% savings; linear time
7. Evaluator-selected input	Any expression chosen by the evaluator	`./build/burs -i`	Correct, verified output on both targets

D.2 Mandatory final submission and demonstration checklist

Status	Item	Where / note
☐	Final presentation following the prescribed structure	**To be prepared (14–18 slides)**
☑	Complete final report in the required format	This document
☑	Stable integrated source code and buildable project package	Repository; `make`
☑	Repository link with history, README, licence/acknowledgements and dependency details	github.com/ahmadazim542/burs-instruction-selector
☐	Installation, build and run instructions verified on the demonstration system	Verified on team laptop; **re-verify on the lab system**
☑	Complete test suite with valid, invalid, boundary, failure and regression cases	`tests/`; Section 9
☑	Result tables, logs, comparison and benchmarking evidence	`results/`; Section 10
☑	Sample inputs, grammars and configuration needed for reproduction	`tests/cases.txt`; `src/targets/*/*.gr`
☑	Final architecture, module, interface and data-flow diagrams matching the implementation	Section 6
☐	Member-wise contribution matrix and contribution log supported by technical evidence	Section 12; **add per-member commit links**
☑	List of known limitations, unsupported cases and future enhancements	Sections 11 and 13

Status	Item	Where / note
☐	Backup copy of code, report, presentation, sample data and demonstration recording	Code and docs on GitHub; **presentation and recording pending**
☑	All external code, libraries, datasets, diagrams and ideas acknowledged	Section 14
☑	No confidential credentials, personal data or unapproved copyrighted material	Only names and register numbers of team members

D.3 Evaluator-selected test record (to be completed during the viva)

Test / scenario	Expected behaviour	Observed output / evidence	Result / evaluator remarks
1. Evaluator-selected input	Valid input: tree, cover and code for both targets, simulation `OK`		
2. Boundary or failure condition	Boundary: correct code; invalid: error with column number, exit status 2		
3. Rebuild / reproducibility check	`make clean && make && make test` builds without warnings and reports 33 passed, 0 failed	Pre-check on team laptop (Apple M1, macOS 26.6, clang 21): 33 passed, 0 failed	

D.4 Review 3 evaluator record

Criterion	Maximum	Marks awarded	Evaluator remarks
Final system completeness and correctness	2.5		
Live demonstration and use-case coverage	1.5		
Testing, results and evaluation	1.5		
Technical depth, design decisions and innovation	1.0		
Final report, repository and reproducibility	1.0		
Verified individual contribution	1.5		
Presentation and viva	1.0		
Total	**10.0**		

Overall final decision: ☐ Completed satisfactorily ☐ Completed with minor corrections ☐ Incomplete / major correction required

Final remarks / corrections / distinctive achievement:

Team Leader Signature Faculty / Evaluator Signature Date